04. Propuesta Técnica — EduAirControl

María de los Ángeles Olaya García; Jhon Jairo Cortez Vargas

Servicio Nacional de Aprendizaje (SENA), Programa ADSO-3145556

Instructora: Karol Daniela Correa

12 de octubre de 2026

04. Propuesta Técnica — EduAirControl

Versión: 1.0

Fecha: 9 de octubre de 2026

Estado: Propuesta técnica basada en el repositorio y la especificación de requisitos disponibles.

Proyecto: EduAirControl

> Criterio de lectura: este documento distingue entre la arquitectura descrita como implementada en Docs-ea-control/05-architecture/overview.md y las recomendaciones o decisiones que requieren validación. La presencia de un componente en la documentación no constituye, por sí sola, evidencia de que esté desplegado y operativo en todos los entornos.

1. Resumen ejecutivo

EduAirControl es una solución web y móvil para administrar información de aulas y sensores y consultar variables ambientales —temperatura, humedad, CO■ y nivel de ruido— en espacios educativos. La propuesta técnica se apoya en una arquitectura de microservicios con una puerta de enlace API, comunicación REST/HTTP para operaciones síncronas y RabbitMQ para eventos asíncronos. PostgreSQL se utiliza para persistencia, Redis para capacidades transversales del gateway y Docker Compose para estandarizar la ejecución de los servicios.

La arquitectura documentada también describe una migración gradual desde un monolito heredado mediante el patrón Strangler Fig. Por ello, el trabajo técnico debe priorizar contratos claros entre servicios, propiedad explícita de los datos, seguridad coherente de extremo a extremo, observabilidad y despliegues reproducibles.

Esta propuesta organiza la solución, recomienda medidas de implementación y señala las decisiones que deben confirmarse antes de considerar la arquitectura lista para producción.

2. Objetivo de la propuesta

Definir una base técnica para implementar, integrar, probar, desplegar y mantener EduAirControl, con atención a:

• separación de responsabilidades por dominio;

• interoperabilidad entre aplicaciones cliente y servicios;

• protección de identidad, permisos y datos;

• recepción confiable de mediciones de sensores;

• evolución controlada desde el sistema heredado;

• mantenibilidad, pruebas y operación.

3. Alcance técnico

3.1 Incluido

• Aplicación web React y aplicación móvil React Native/Expo.

• Spring Cloud Gateway como punto de entrada de las solicitudes de backend.

• Servicios de seguridad, gestión de usuarios, aulas, sensores, monitoreo ambiental y experiencia de usuario.

• PostgreSQL como motor relacional.

• RabbitMQ para publicación y consumo de eventos de dominio.

• Redis para funciones transversales descritas en la arquitectura.

• Ingesta HTTP de dispositivos, con X-API-Key por dispositivo, según la arquitectura documentada.

• Contenedores Docker y orquestación de desarrollo mediante Docker Compose.

• Pruebas, configuración por entorno, documentación de API y prácticas de seguridad.

3.2 Fuera del alcance de esta propuesta

• Compra, instalación física o calibración certificada de sensores.

• Garantizar precisión metrológica sin especificaciones del fabricante y procedimientos de calibración.

• Alta disponibilidad multi-región o infraestructura cloud concreta, ya que no se identifica un proveedor objetivo confirmado.

• Presentar MQTT como mecanismo implementado: la arquitectura consultada documenta ingesta HTTP; cualquier uso de MQTT requiere decisión y evidencia adicionales.

• Declarar que todos los servicios están listos para producción sin una auditoría de código, configuración y despliegue.

4. Arquitectura técnica propuesta

4.1 Estilo y principios

El estilo documentado es microservicios + API Gateway + comunicación basada en eventos. La migración se describe como incremental, desde un monolito legado hacia servicios delimitados por contexto.

Principios recomendados:

• Responsabilidad única por servicio: cada servicio posee su lógica y datos de dominio.

• Contratos explícitos: las APIs y eventos se versionan y documentan.

• Propiedad de datos: un servicio no consulta directamente las tablas internas de otro.

• Seguridad por capas: autenticación en el punto de entrada y autorización también en el servicio propietario.

• Resiliencia: tiempos de espera, reintentos limitados, idempotencia y manejo de fallos.

• Configuración externa: secretos y parámetros se suministran mediante configuración segura, no se codifican en el repositorio.

• Trazabilidad: las solicitudes y eventos deben correlacionarse mediante identificadores.

• Evolución gradual: retirar rutas del monolito solo después de validar equivalencia funcional y datos.

4.2 Vista lógica de componentes

• | Componente | Responsabilidad técnica | Tecnología documentada |

• |---|---|---|

• | Cliente web | Interfaz administrativa y consulta de información | React |

• | Cliente móvil | Acceso desde dispositivos móviles | React Native / Expo |

• | API Gateway | Entrada de backend, enrutamiento, validación JWT, limitación de tasa e identificadores de correlación | Spring Cloud Gateway |

• | ms-security | Emisión y administración de identidad/tokens | Spring Security, JWT, OAuth2/OIDC descritos |

• | ms-user-management | Gestión del dominio de usuarios | Microservicio independiente |

• | ms-classroom-management | Gestión de aulas | Microservicio independiente |

• | ms-sensor-management | Gestión y registro de sensores | Microservicio independiente |

• | ms-environment-monitoring | Recepción/consulta de mediciones y lógica de monitoreo; absorbe el servicio de alertas retirado | Microservicio + eventos |

• | ms-user-experience | Funciones del dominio de experiencia de usuario | Microservicio independiente |

• | PostgreSQL | Persistencia relacional | PostgreSQL |

• | RabbitMQ | Distribución asíncrona de eventos | Topic exchange y consumidores idempotentes descritos |

• | Redis | Limitación de tasa, sesiones, lista de revocación y contadores de bloqueo | Redis |

• | Docker Compose | Ejecución local coordinada | Docker |

Nota de validación: la arquitectura declara seis instancias PostgreSQL, pero indica que ms-environment-monitoring y ms-user-experience todavía usan la base eduaircontrol con

esquemas separados. Se recomienda conservar esta situación como deuda técnica identificada y decidir una separación completa con migraciones verificadas, sin asumir que ya existe aislamiento físico para todos los servicios.

4.3 Comunicación

Síncrona: cliente → API Gateway → servicio responsable, mediante REST/HTTP. En producción, el tráfico externo debe utilizar HTTPS. Las rutas internas y los puertos deben permanecer en la red privada de servicios siempre que sea posible.

Asíncrona: los servicios publican eventos mediante un patrón transactional outbox y RabbitMQ; los consumidores deben ser idempotentes porque la entrega puede ocurrir más de una vez. Cada evento debería incluir identificador único, tipo, versión de esquema, fecha de creación, productor y correlación.

Dispositivos IoT: la documentación consultada define ingesta HTTP con cabecera X-API-Key individual por dispositivo. Deben validarse identidad del dispositivo, formato, límites físicos plausibles, marca temporal, duplicados y frecuencia de envío. La clave debe poder rotarse y revocarse. No debe tratarse la API key como sustituto del cifrado de transporte.

5. Tecnologías y justificación

• | Capa | Tecnología base documentada | Justificación / criterio |

• |---|---|---|

• | Web | React | Interfaz web por componentes |

• | Móvil | React Native / Expo | Reutilización de capacidades móviles y flujo de desarrollo Expo |

• | Backend | Spring Cloud Gateway y microservicios del ecosistema Spring descritos en el repositorio | Enrutamiento, integración con seguridad y separación por dominio |

• | Identidad | Spring Security, JWT y referencias a OAuth2/OIDC | Autenticación y autorización; requiere uniformar el esquema entre gateway y servicios |
• | Persistencia | PostgreSQL | Modelo relacional y consistencia transaccional |
• | Mensajería | RabbitMQ | Desacoplamiento de eventos entre dominios |
• | Caché/estado transversal | Redis | Rate limiting, sesiones y listas/contadores de seguridad documentados |
• | Contenedores | Docker / Docker Compose | Entornos repetibles para desarrollo e integración |
• | Ingesta de sensores | HTTP con X-API-Key | Mecanismo que documenta la arquitectura actual |

Las versiones exactas de todos los componentes, las políticas de soporte y las imágenes base deben fijarse en archivos de construcción y revisarse antes de liberar una versión.

6. Diseño de servicios y límites de dominio

6.1 Catálogo propuesto

• Seguridad (ms-security): autenticación, emisión de tokens y capacidades relacionadas con identidad.
• Usuarios (ms-user-management): altas, consultas, cambios y estado de usuarios.
• Aulas (ms-classroom-management): datos de aulas, ubicación, capacidad y estado.
• Sensores (ms-sensor-management): registro, asignación, identificación y estado de dispositivos.
• Monitoreo ambiental (ms-environment-monitoring): ingestión y consulta de lecturas, evaluación frente a umbrales configurados y eventual gestión de alertas.
• Experiencia de usuario (ms-user-experience): funciones del dominio de experiencia, según los contratos reales del repositorio.

Los nombres describen los límites que figuran en la arquitectura. La responsabilidad final de cada endpoint debe confirmarse contra el código y el catálogo de APIs; no se deben duplicar reglas de negocio entre servicios.

6.2 Propiedad de datos

Cada servicio debe ser el propietario de sus entidades y exponer los datos a otros servicios mediante APIs o eventos. Las lecturas entre servicios no deben realizarse directamente contra tablas ajenas. Para el estado compartido actual de base de datos, se recomienda:

• documentar qué esquema y tablas pertenecen a cada servicio;
• restringir credenciales de base de datos por servicio;
• medir dependencias existentes antes de separarlas;
• migrar datos con validación de conteos, integridad y posibilidad de reversión;
• retirar permisos y conexiones heredadas después de verificar el cambio.

6.3 Datos ambientales

Cada lectura debería identificar, como mínimo, el dispositivo o sensor, la variable, el valor y unidad, la marca temporal de medición, la marca temporal de recepción y la referencia del aula o ubicación. La estructura exacta debe coincidir con el modelo real de datos y las APIs del proyecto.

Los umbrales de temperatura, humedad, CO■ y ruido deben ser configurables y tener fuente, unidad, ámbito de aplicación y versión. Los rangos de la especificación de requisitos no deben presentarse como estándares universales ni sustituir normas locales, orientación profesional o calibración del dispositivo.

7. Seguridad

7.1 Identidad y autorización

La documentación arquitectónica describe que ms-security emite JWT RS256, mientras algunos servicios validan HS256 o confían en cabeceras internas como X-User-Id, X-User-Role y X-Institution-Id durante una etapa intermedia. Esta diferencia es un riesgo de consistencia.

Recomendación prioritaria: definir y aprobar un único contrato de identidad entre gateway y servicios. Como mínimo:

• documentar algoritmo, emisor, audiencia, expiración, claves y rotación;
• validar firma, iss, aud, exp y demás claims requeridos;
• evitar aceptar cabeceras de identidad desde clientes externos;
• permitir que el gateway elimine y regenere las cabeceras internas confiables;
• mantener autorización por rol y ámbito institucional en el servicio propietario;
• probar casos 401/403 y accesos cruzados entre instituciones;
• diseñar revocación, expiración y renovación de tokens.

No se debe cambiar el algoritmo o formato de tokens sin migración coordinada y pruebas de compatibilidad.

7.2 Secretos y configuración

• No incluir secretos reales en Git, imágenes, documentación pública ni archivos de ejemplo.
• Proporcionar .env.example con valores ficticios y validación de variables obligatorias.
• Usar secretos separados por entorno y permisos mínimos.
• Rotar credenciales ante exposición y documentar el procedimiento.
• No usar valores por defecto inseguros en producción.
• Separar configuración de desarrollo, pruebas y producción.

7.3 Protección de API y dispositivos

• Aplicar límites de tasa y tamaños máximos de petición.

• Validar datos en la frontera y también en el servicio propietario.

• Registrar intentos de autenticación y eventos relevantes sin exponer contraseñas, tokens ni claves.

• Usar API keys únicas por sensor, con almacenamiento seguro, rotación y revocación.

• Añadir protección contra repetición o duplicados en la ingesta cuando aplique.

• Revisar CORS para permitir solo orígenes conocidos en cada entorno.

• Mantener dependencias e imágenes actualizadas y analizar vulnerabilidades.

8. Despliegue e infraestructura

8.1 Desarrollo e integración

Docker Compose es la herramienta de ejecución local documentada. La configuración debe incluir servicios, redes, volúmenes persistentes para datos que deban conservarse, comprobaciones de salud y dependencias de arranque. Un servicio no debe considerarse listo únicamente porque el contenedor esté iniciado; debe superar su health check.

8.2 Entornos

• | Entorno | Uso | Requisitos mínimos |

• |---|---|---|

• | Desarrollo | Trabajo individual e integración temprana | Datos ficticios, logs útiles, configuración local |

• | Pruebas/CI | Validación reproducible de cambios | Pruebas automáticas, bases aisladas, configuración efímera |

• | Preproducción | Ensayo de despliegue y aceptación | Configuración similar a producción, sin datos sensibles innecesarios |

• | Producción | Uso real | HTTPS, secretos gestionados, copias de seguridad, monitoreo, control de acceso y recuperación probada |

La estrategia concreta de despliegue productivo aún debe seleccionarse. No se propone un proveedor cloud específico sin requisitos de presupuesto, disponibilidad, residencia de datos y operación.

8.3 Pipeline recomendado

• Validar formato, lint y compilación.

• Ejecutar pruebas unitarias.

• Ejecutar pruebas de integración con dependencias aisladas.

• Validar contratos REST y eventos.

• Analizar dependencias y contenedores.

• Construir imágenes etiquetadas de forma inmutable.

• Desplegar en entorno de pruebas.

• Ejecutar comprobaciones de salud y pruebas de humo.

• Aprobar la promoción a preproducción/producción.

• Mantener un procedimiento de reversión.

Este pipeline es una recomendación; no se afirma que todos sus pasos ya estén implementados.

9. Calidad, pruebas y aceptación técnica

Se recomienda una pirámide de pruebas equilibrada:

• Unitarias: reglas de dominio, validaciones y transformaciones.

• Integración: persistencia, migraciones, seguridad y mensajería.

• Contrato: compatibilidad de APIs y eventos entre productor y consumidor.

• End-to-end: flujos críticos desde web/móvil hasta los servicios.

• Resiliencia: caída de RabbitMQ/Redis, tiempos de espera, reintentos y recuperación.

• Seguridad: autorización, exposición de secretos, abuso de API e ingreso de dispositivos.

Criterios mínimos de aceptación técnica

• Los servicios arrancan con configuración documentada y verificaciones de salud.

• Los endpoints tienen contratos documentados, códigos de error consistentes y validación de entrada.

• Las rutas protegidas rechazan usuarios no autenticados y roles no autorizados.

• Los eventos soportan duplicados sin producir efectos de negocio duplicados.

• Las migraciones de base de datos se ejecutan de manera reproducible.

• Las lecturas de sensores se validan y pueden rastrearse hasta el dispositivo.

• Los registros permiten correlacionar una solicitud entre gateway y servicios.

• Existe una copia de seguridad y se ha probado la restauración.

• Las dependencias críticas y vulnerabilidades tienen revisión antes de liberar.

Los umbrales cuantitativos de rendimiento y disponibilidad deben definirse a partir de carga esperada y necesidades institucionales; no se fijan cifras arbitrarias en esta propuesta.

10. Observabilidad y operación

La arquitectura menciona identificadores de correlación y componentes transversales, pero el despliegue completo de una plataforma de observabilidad no queda confirmado. Se recomienda definir:

• logs estructurados con traceId/correlationId, servicio, entorno y resultado;

• métricas de latencia, tasa de errores, tráfico, saturación, colas y salud de base de datos;

• alertas por fallos sostenidos, acumulación de mensajes, ausencia de datos de sensores y errores de autenticación;

• paneles operativos y guía de respuesta a incidentes;

• políticas de retención y acceso a registros, evitando datos sensibles;

• objetivos de servicio (SLO) aprobados por los responsables.

11. Plan de implementación recomendado

• | Fase | Actividades | Resultado verificable |

• |---|---|---|

• | 1. Confirmación | Inventario de servicios, APIs, variables, dependencias y brechas entre documentación y código | Matriz de estado real y decisiones pendientes |

• | 2. Fundamentos | Contratos de seguridad, configuración, health checks y estándares de API | Configuración y contratos aprobados |

• | 3. Dominios principales | Validación de usuarios, aulas y sensores; persistencia y migraciones | Flujos de gestión probados |

• | 4. Monitoreo | Ingesta HTTP, validación de lecturas, reglas de umbral y eventos | Lecturas trazables y pruebas de idempotencia |

• | 5. Integración cliente | Conectar web y móvil a las rutas del gateway y probar errores de sesión | Flujos críticos de cliente validados |

• | 6. Calidad y seguridad | Pruebas automatizadas, revisión de dependencias, permisos y secretos | Informe de resultados y correcciones |

• | 7. Preparación operativa | Copias, restauración, métricas, manuales y reversión | Lista de preparación para despliegue |

• | 8. Liberación gradual | Desplegar primero en entorno controlado, comparar resultados y migrar rutas | Evidencia de aceptación y plan de reversión |

Las fases son una secuencia sugerida; la duración y el orden definitivo deben ajustarse a la situación real del repositorio y del equipo.

12. Riesgos y mitigaciones

• | Riesgo | Impacto | Mitigación propuesta |

• |---|---|---|

• | Esquemas de JWT distintos entre gateway y servicios | Rechazos, permisos inconsistentes o exposición de rutas | Contrato único, pruebas de integración y migración coordinada |

• | Servicios que comparten una base física | Acoplamiento y despliegues más riesgosos | Propiedad por esquema, credenciales mínimas y plan de separación |

• | Documentación genérica o desactualizada | Decisiones basadas en supuestos incorrectos | Verificar cada afirmación contra código, configuración y pruebas |

• | Eventos duplicados o fuera de orden | Datos o efectos repetidos | Idempotencia, identificadores únicos y estrategia de reintentos |

• | Credenciales de dispositivo expuestas | Ingesta no autorizada | Claves únicas, rotación, revocación y transporte cifrado |

• | Umbrales ambientales sin validación contextual | Alertas inadecuadas | Umbrales configurables, fuente documentada y aprobación institucional |

• | Falta de recuperación probada | Pérdida prolongada de servicio/datos | Copias de seguridad y simulacros de restauración |

• | Migración del monolito incompleta | Funciones duplicadas o inconsistentes | Migración por ruta, comparación y plan de reversión |

13. Decisiones que requieren confirmación

Antes de cerrar el diseño definitivo, el equipo debe confirmar:

• El contrato final de JWT: RS256/JWKS frente a validación HS256 y uso de cabeceras internas.

• El estado real y las versiones de cada servicio y dependencia.

• La estrategia de separación de bases de datos y esquemas compartidos.

• Los contratos definitivos de APIs y eventos, incluidos versionado y compatibilidad.

• La fuente oficial y la configuración de umbrales ambientales.

• El entorno de despliegue productivo, disponibilidad esperada y residencia de datos.

• Los objetivos medibles de rendimiento, retención, recuperación y disponibilidad.

• Si se requiere MQTT en el futuro; no debe darse por implementado con la evidencia actual.

• Los requisitos institucionales y legales aplicables al tratamiento de datos de usuarios y mediciones.

14. Entregables técnicos derivados

• Diagrama de contexto y contenedores C4 actualizado.

• Catálogo de microservicios, responsables y dependencias.

• Contratos REST y eventos versionados.

• Modelo de datos y matriz de propiedad por servicio.

• Configuración reproducible de desarrollo y CI.

• Suite de pruebas automatizadas e informe de resultados.

• Guía de despliegue, reversión, copias y recuperación.

• Manual de operación, monitoreo y respuesta a incidentes.

• Matriz de riesgos y decisiones de arquitectura (ADR).

Presupuesto y tiempo de desarrollo

15. Tiempo de desarrollo del proyecto

El proyecto EduAirControl tuvo un periodo de trabajo aproximado de 18 meses, durante el cual se realizaron actividades de análisis, diseño, desarrollo, integración, pruebas y documentación.

Sin embargo, se estima que el proyecto podría desarrollarse en un periodo de 4 a 5 meses de trabajo intensivo, considerando una dedicación de 8 horas diarias. Esta diferencia se debe a que los 18 meses corresponden al tiempo transcurrido durante la elaboración del proyecto y no necesariamente a una dedicación laboral continua de tiempo completo.

• | Concepto | Estimación |

• | ---------- | ----------: |

• | Tiempo transcurrido durante el proyecto | 18 meses |

• | Tiempo estimado de desarrollo intensivo | 4 a 5 meses |

• | Jornada de referencia | 8 horas diarias |

• | Costo estimado del desarrollo del software | $20.000.000 COP |

El costo del desarrollo representa una estimación del esfuerzo de análisis, programación, integración, pruebas y documentación. No corresponde necesariamente a dinero efectivamente pagado, por lo que debe presentarse como costo estimado si el equipo no recibió esa remuneración.

16. Presupuesto de hardware IoT

Para desarrollar y probar el componente de monitoreo ambiental se contemplan dos computadores y los dispositivos electrónicos necesarios para realizar las mediciones de las variables ambientales.

• | Categoría | Cantidad | Precio unitario | Costo total |

• | ---------- | -------: | ----------: | ----------: |

• | Computadores portátiles | 2 | $5.500.000 | $11.000.000 |

• | Mouse | 2 | $160.000 | $320.000 |

• | Placa de desarrollo ESP32 | 1 | $40.000 | $40.000 |

• | Sensor de temperatura y humedad DHT11 | 1 | $8.000 | $8.000 |

• | Sensor/módulo de sonido KY-038 | 1 | $8.000 | $8.000 |

• | Sensor de calidad del aire MQ-135 | 1 | $14.000 | $14.000 |

• | Total de hardware | | | $11.390.000 |

Observación técnica: el DHT11 mide temperatura y humedad relativa. El KY-038 permite detectar sonido y obtener un nivel relativo mediante su salida analógica; no debe interpretarse como un sonómetro calibrado en dB sin un procedimiento de calibración. El MQ-135 permite detectar cambios en la concentración de determinados gases, pero no ofrece por sí solo una medición precisa y selectiva de CO■ en ppm. Por ello, las especificaciones del hardware deben coincidir con los dispositivos que realmente se utilizaron.

17. Gastos de electricidad e internet

Durante los 18 meses de trabajo también se consideran los gastos de electricidad e internet necesarios para programar, realizar pruebas, consultar documentación, gestionar el repositorio y conectar los dispositivos.

Como no se cuenta aquí con las facturas reales, se utiliza el siguiente presupuesto de referencia:

• | Concepto | Costo mensual estimado | Periodo | Costo estimado |

• | ---------- | ----------: | ----------: | ----------: |

• | Electricidad atribuible al proyecto | $80.000 | 18 meses | $1.440.000 |

• | Servicio de internet | $120.000 | 18 meses | $2.160.000 |

• | Total de servicios | $200.000 | 18 meses | $3.600.000 |

Estos valores son estimaciones presupuestales, no gastos comprobados. Para presentar el costo real, deben reemplazarse por los valores efectivamente pagados o por una proporción razonable de las facturas atribuible al proyecto.

18. Presupuesto general

Al integrar el costo estimado del desarrollo, el hardware y los servicios utilizados durante el proyecto, se obtiene el siguiente presupuesto:

• | Concepto | Valor |

• | ---------- | ----------: |

• | Desarrollo del software | $20.000.000 |

• | Hardware para el desarrollo y las pruebas | $11.390.000 |

• | Electricidad durante 18 meses | $1.440.000 |

• | Internet durante 18 meses | $2.160.000 |

• | Presupuesto total estimado | $34.990.000 COP |

19. Consideraciones del presupuesto

El presupuesto total estimado de EduAirControl es de $34.990.000 COP. Este valor incluye el costo de referencia del desarrollo del software, los equipos informáticos, los componentes electrónicos IoT y los gastos estimados de electricidad e internet.

El presupuesto no debe interpretarse automáticamente como dinero desembolsado. El costo de desarrollo es una valoración del trabajo realizado, mientras que los equipos y servicios pueden representar compras o gastos reales si se dispone de los comprobantes correspondientes.

Los valores también pueden variar de acuerdo con los precios de mercado, los equipos finalmente adquiridos, el consumo energético, el plan de internet y el tiempo efectivo de

dedicación del equipo.

20. Conclusión

La base técnica de EduAirControl apunta a una arquitectura modular y extensible que combina clientes web/móvil, microservicios, gateway, PostgreSQL, RabbitMQ, Redis y contenedores. Para convertir esa base en una solución verificablemente estable, las prioridades son uniformar el contrato de seguridad, reforzar la propiedad de datos, documentar APIs/eventos, automatizar pruebas, validar la ingesta de sensores y cerrar los aspectos de despliegue y recuperación.

Esta propuesta es una guía de diseño e implementación basada en los artefactos disponibles. Las decisiones marcadas como pendientes deben resolverse con evidencia del código, pruebas y necesidades reales del entorno antes de certificar el sistema como listo para producción.

21. Fuentes internas consultadas

• Docs-ea-control/05-architecture/overview.md — arquitectura, servicios, comunicación y tecnologías.
• Docs-ea-control/05-architecture/README.md — decisiones de arquitectura y estado de ADR.
• Docs-ea-control/09-microservices/README.md — organización de la documentación de microservicios.
• Docs-ea-control/12-ux-ui/design-system.md — lineamientos de interfaz; se trata como guía de diseño, no como prueba de implementación.
• SRS_EduAirControl suministrado por el equipo — alcance y requisitos del sistema.